ANKARA ÜNİVERSİTESİ

MÜHENDİSLİK FAKÜLTESİ

BİLGİSAYAR MÜHENDİSLİĞİ BÖLÜMÜ

BLM4522 - Ağ Tabanlı Paralel Dağıtım Sistemleri

Proje 2. Veritabanı Yedekleme ve Felaketten Kurtarma Planı

Proje 5. Veri Temizleme ve ETL Süreçleri Tasarımı

Açıklama Videoları

https://www.youtube.com/watch?v=N4cU7T-tW7Q

https://www.youtube.com/watch?v=A68-aP6F_Is

Github Bağlantısı

https://github.com/TimurMalkc/BLM4522

Timur Malkoç – 22290719

İÇİNDEKİLER

İÇİNDEKİLER 2
1. VERİTABANI YEDEKLEME VE FELAKETTEN KURTARMA PLANI 3
1.1. Giriş 3
1.2. Sistem ve Veritabanı 3
1.3. Yedekleme Stratejileri 4
1.4. Felaket Senaryoları ve Veri Kurtarma 5
1.5. Point-In-Time Kurtarma 6
1.6. Zamanlayıcı ile Otomatik Yedekleme 7
1.7. Yedeklerin Doğruluğunun Test Edilmesi 7
1.8. Data Mirroring 7
1.9. Sonuç 8
2. VERİ TEMİZLEME VE ETL SÜREÇLERİ TASARIMI 9
2.1. Giriş 9
2.2. Veri Seti ve Ortam Hazırlığı (Extract) 9
2.3. Staging Katmanı Oluşturma 9
2.4. Veri Dönüştürme (Transform) 10
2.5. Veri Yükleme (Load) 11
2.6. Veri Temizleme ve Kalite Analizi 11
2.7. Karşılaştırmalı Raporlama 12
2.8. Sonuç 13

1. VERİTABANI YEDEKLEME VE FELAKETTEN KURTARMA PLANI

1.1 Giriş

Bu çalışmada, veritabanı sistemlerinde veri güvenliğini sağlamak amacıyla kullanılan yedekleme ve felaketten kurtarma yöntemleri ele alınmıştır. Proje kapsamında, Microsoft SQL Server kullanılarak bir veritabanı oluşturulmuş ve bu veritabanı üzerinde farklı yedekleme stratejileri uygulanmıştır.

Çalışmanın temel amacı; veri kaybı durumlarında sistemin en kısa sürede ve en doğru şekilde eski haline döndürülebilmesini sağlayan yöntemleri uygulamalı olarak göstermektir. Bu doğrultuda tam (full), fark (differential) ve artık (transaction log) yedekleme türleri incelenmiş, felaket senaryoları oluşturularak veri kurtarma süreçleri test edilmiştir.

Ayrıca, yedekleme işlemlerinin otomatik hale getirilmesi, yedeklerin doğruluğunun test edilmesi ve veri sürekliliğini sağlamak amacıyla kullanılan ileri düzey teknikler de değerlendirilmiştir.

1.2. Sistem ve Veritabanı

Proje kapsamında Microsoft SQL Server kullanılmıştır. Sunucu ortamı local olarak yapılandırılmıştır. Çalışma kapsamında “mainDatabase” adlı bir veritabanı oluşturulmuştur. Veritabanı üzerinde işlemlerin gerçekleştirilebilmesi için query ekranları kullanılmıştır.

Veritabanı içerisinde “Customers” adlı bir tablo oluşturulmuş ve bu tabloya kimlik (id), isim (name) ve e-posta (email) alanları eklenmiştir. Hazır veri seti kullanmak yerine, sistem üzerindeki değişikliklerin daha net gözlemlenebilmesi amacıyla 55 kayıt içeren bir veri seti manuel olarak oluşturulmuştur. Oluşturulan veriler tabloya eklenmiş ve doğrulama işlemleri gerçekleştirilmiştir.

Results | Messages

	Id	Name	Email	CreatedDate
1	1	Ali	ali@test.com	2026-04-20 16:27:56.840
2	2	Veli	veli@test.com	2026-04-20 16:27:56.840
3	3	Ayse	ayse@test.com	2026-04-20 16:27:56.840
4	4	Fatma	fatma@test.com	2026-04-20 16:27:56.840
5	5	Mehmet	mehmet@test.com	2026-04-20 16:27:56.840
6	6	Ahmet	ahmet@test.com	2026-04-20 16:27:56.840
7	7	Mustafa	mustafa@test.com	2026-04-20 16:27:56.840
8	8	Hasan	hasan@test.com	2026-04-20 16:27:56.840
9	9	Huseyin	huseyin@test.com	2026-04-20 16:27:56.840
10	10	Ibrahim	ibrahim@test.com	2026-04-20 16:27:56.840
11	11	Yusuf	yusuf@test.com	2026-04-20 16:27:56.840
12	12	Osman	osman@test.com	2026-04-20 16:27:56.840
13	13	Emre	emre@test.com	2026-04-20 16:27:56.840
14	14	Mehmet Ali	mehmetali@test.com	2026-04-20 16:27:56.840
15	15	Zeynep	zeynep@test.com	2026-04-20 16:27:56.840
16	16	Elif	elif@test.com	2026-04-20 16:27:56.840

Şekil 1.1.1. “Customers” adlı tablo

1.3. Yedekleme Stratejileri

Proje kapsamında üç temel yedekleme yöntemi ele alınmıştır:

- **Tam Yedekleme (Full Backup):** Veritabanının tamamının kopyası alınmıştır. Bu yöntem temel yedekleme türü olarak kullanılmıştır.
- **Fark Yedekleme (Differential Backup):** Son tam yedeklemeden sonra değişen veriler yedeklenmiştir.
- **Artık Yedekleme (Transaction Log Backup):** Son log yedeğinden itibaren yapılan tüm değişiklikler kaydedilmiştir.

İlk aşamada mevcut veri seti için tam yedekleme gerçekleştirilmiştir. Daha sonra veri seti üzerinde değişiklik yapılarak diferansiyel yedekleme alınmıştır. Ek olarak işlem günlüğü yedekleme işlemleri de uygulanmıştır.

```sql
BACKUP DATABASE [mainDatabase]
TO DISK = 'C:\SQLBackup\mainDatabase_Full.bak'
WITH FORMAT, MEDIANAME = 'SQLServerBackups', NAME = 'Full Backup of mainDatabase';

BACKUP DATABASE [mainDatabase]
TO DISK = 'C:\SQLBackup\mainDatabase_Diff.bak'
WITH DIFFERENTIAL;

BACKUP LOG [mainDatabase]
TO DISK = 'C:\SQLBackup\mainDatabase_Log.trn'
WITH FORMAT;
```

Şekil 1.2.1. Yedekleme yöntemlerinin kodları

1.4. Felaket Senaryoları ve Veri Kurtarma

Felaket senaryosu oluşturmak amacıyla “Customers” tablosundaki tüm veriler silinmiştir.

```sql
DELETE FROM mainDatabase.dbo.Customers;
```

Şekil 1.3.1. Felaket senaryosu için veri silme kodu

a) Tam Yedekten Kurtarma: Veritabanı kurtarma işlemi öncesinde gerekli yetkilerin sağlanabilmesi için sistem “master” moduna alınmış ve kullanıcı erişim hatalarının önüne geçmek amacıyla “single-user” moduna geçirilmiştir.

Tam yedek kullanılarak veri geri yükleme işlemi gerçekleştirilmiş ve işlem sonrasında sistem tekrar “multi-user” moduna alınmıştır. Yapılan kontrol sonucunda verilerin geri yüklendiği gözlemlenmiştir. Ancak tam yedeklemeden sonra eklenen verilerin geri gelmediği tespit edilmiştir.

```sql
USE [master];
GO

ALTER DATABASE [mainDatabase] SET SINGLE_USER WITH ROLLBACK IMMEDIATE;
GO

RESTORE DATABASE [mainDatabase]
FROM DISK = 'C:\SQLBackup\mainDatabase_Full.bak'
WITH REPLACE;
GO

ALTER DATABASE [mainDatabase] SET MULTI_USER;
GO
```

Şekil 1.3.2. Tam yedek ile veri kurtarma

b) Fark Yedekten Kurtarma: Kayıp verilerin geri getirilebilmesi için tam yedek ile fark yedek kullanılmıştır. Fark yedek sadece değiştirilen verileri tutabildiği için tek başına bütün verileri getirmek için yeterli değildir ve tam yedek ile kullanılması gerekmektedir.

Geri yükleme işlemi sırasında “NORECOVERY” ve “RECOVERY” seçenekleri kullanılarak veritabanının restore süreci kontrol altına alınmıştır. İşlem sonucunda eksik verilerin başarıyla geri getirildiği doğrulanmıştır.

```sql
USE [master];
GO
ALTER DATABASE [mainDatabase] SET SINGLE_USER WITH ROLLBACK IMMEDIATE;
GO

RESTORE DATABASE [mainDatabase]
FROM DISK = 'C:\SQLBackup\mainDatabase_Full.bak'
WITH REPLACE, NORECOVERY;

RESTORE DATABASE [mainDatabase]
FROM DISK = 'C:\SQLBackup\mainDatabase_Diff.bak'
WITH RECOVERY;

ALTER DATABASE [mainDatabase] SET MULTI_USER;
GO
```

Şekil 1.3.3. Fark yedek ile veri kurtarma

1.5 Point-In-Time Kurtarma

Fark (transaction log) yedekleme kullanılarak belirli bir zamana geri dönme (point-in-time restore) işlemi gerçekleştirilmiştir.

Bu kapsamda tam yedek alınmıştır, veri üzerinde değişiklik yapılmış ve zaman bilgisi kaydedilmiştir, veri silinmiş ve işlem günlüğü yedeği alınmıştır.

Kurtarma aşamasında, tam yedek geri yüklenmiş ve ardından “STOPAT” parametresi kullanılarak belirli bir zamana kadar olan veriler geri getirilmiştir.

Bu yöntem sayesinde veritabanı, istenilen zamandaki durumuna başarıyla döndürülmüştür.

```sql
BACKUP DATABASE [mainDatabase] TO DISK = 'C:\SQLBackup\mainDatabase_Full.bak' WITH FORMAT;
INSERT INTO mainDatabase.dbo.Customers VALUES (58, 'Ahmet', 'ahmet@test.com',GETDATE());
SELECT GETDATE() AS 'TargetTime';
DELETE FROM mainDatabase.dbo.Customers;
BACKUP LOG [mainDatabase] TO DISK = 'C:\SQLBackup\mainDatabase_Log.trn' WITH FORMAT;


USE [master];
GO
ALTER DATABASE [mainDatabase] SET SINGLE_USER WITH ROLLBACK IMMEDIATE;
GO

RESTORE DATABASE [mainDatabase]
FROM DISK = 'C:\SQLBackup\mainDatabase_Full.bak'
WITH REPLACE, NORECOVERY;

RESTORE LOG [mainDatabase]
FROM DISK = 'C:\SQLBackup\mainDatabase_Log.trn'
WITH RECOVERY,
STOPAT = '2026-04-20 12:45:05.987';

ALTER DATABASE [mainDatabase] SET MULTI_USER;
GO
```

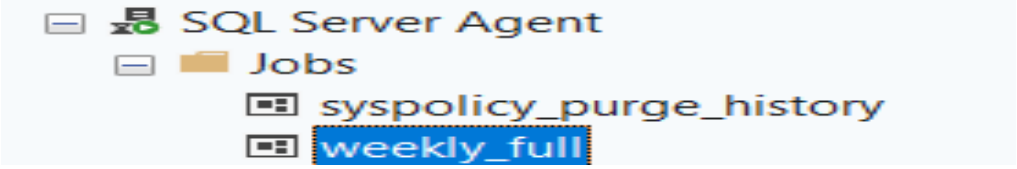
Şekil 1.4.1. Point-in-time kurtarma işlemi

1.6. Zamanlayıcı ile Otomatik Yedekleme

Yedekleme işlemlerinin otomatik hale getirilmesi amacıyla SQL Server Agent kullanılmıştır.

Bu kapsamda SQL Server Agent servisi aktif hale getirilmiştir, yeni bir job oluşturulmuştur, haftalık tam yedekleme görevi tanımlanmıştır; görev, her hafta pazar günü gece yarısı çalışacak şekilde zamanlanmıştır.

Oluşturulan job, manuel olarak tetiklenmiş ve yedekleme işleminin başarıyla gerçekleştiği doğrulanmıştır.



Şekil 1.5.1. Görevi haftalık gerçekleştiren job

1.7. Yedeklerin Doğruluğunun Test Edilmesi

Yedeklerin güvenilirliğini test etmek amacıyla üç farklı yöntem uygulanmıştır:

- **VERIFYONLY:** Yedek dosyasının bozuk olup olmadığı kontrol edilmiştir.
- **CHECKSUM:** Yedekleme sırasında veri bütünlüğü doğrulaması sağlanmıştır.
- **Sandbox Ortamı:** Yedek, farklı bir test veritabanına yüklenerek doğrulama yapılmıştır.

Bu yöntemler sayesinde yedeklerin kullanılabilir ve güvenilir olduğu doğrulanmıştır.

```sql
RESTORE VERIFYONLY
FROM DISK = 'C:\SQLBackup\mainDatabase_Full.bak';
GO

BACKUP DATABASE [mainDatabase] TO DISK = 'C:\Backups\mainDatabase_Full_Secure.bak'
WITH CHECKSUM, CONTINUE_AFTER_ERROR;

USE [master]; GO RESTORE DATABASE [mainDatabase_Test] FROM DISK = 'C:\Backups\mainDatabase_Full.bak'WITH
MOVE 'mainDatabase' TO 'C:\Backups\mainDatabase_Test_Data.mdf',
MOVE 'mainDatabase_log' TO 'C:\Backups\mainDatabase_Test_Log.ldf', REPLACE; GO
```

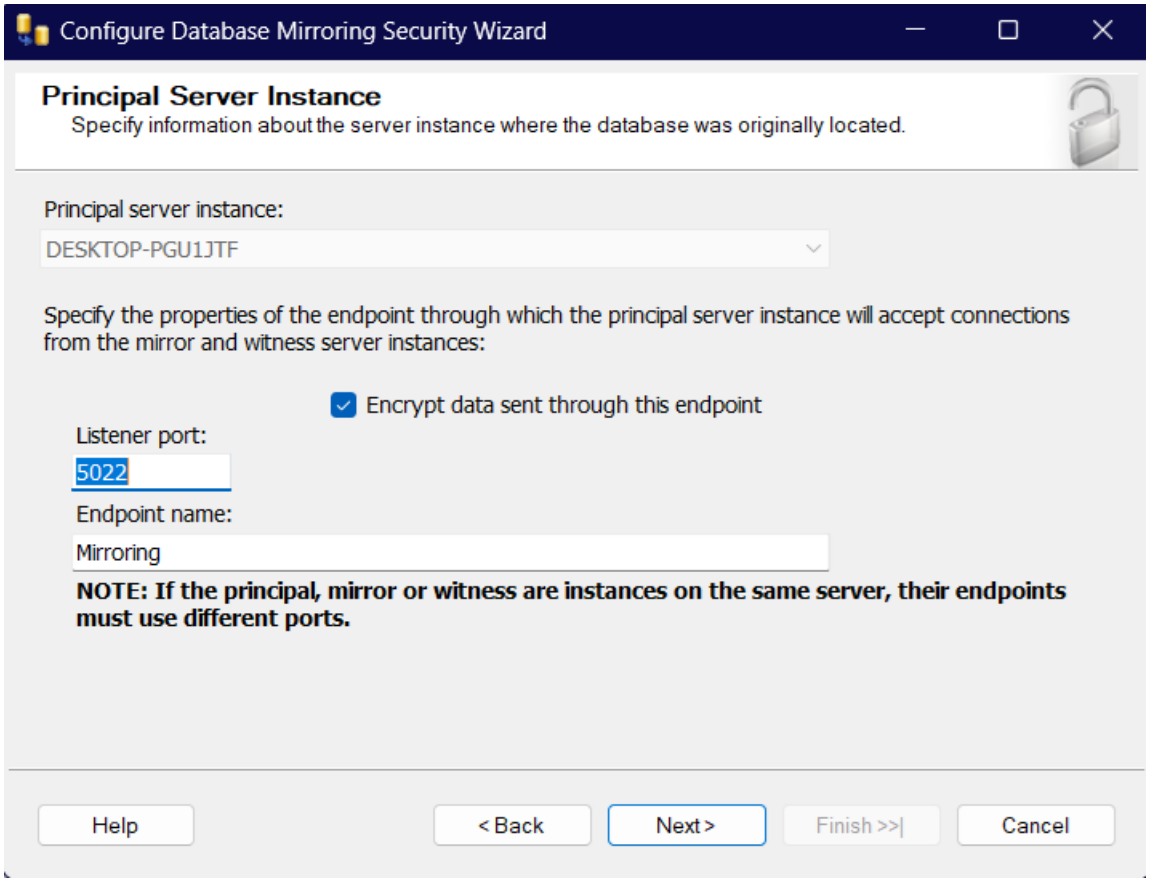
Şekil 1.6.1. Group API grupları

1.8. Data Mirroring

Veri sürekliliğini sağlamak amacıyla veri yansılama (database mirroring) yöntemi incelenmiştir.

Bu yöntemde, ana veritabanının gerçek zamanlı bir kopyası ikinci bir sunucuda tutulmaktadır. Ana sistemde oluşabilecek bir arıza durumunda sistem otomatik olarak yedek sunucuya geçiş yapabilmektedir.

Donanımsal kısıtlar nedeniyle uygulama tam olarak gerçekleştirilememiş, ancak yapılandırma adımları incelenmiş ve teorik olarak açıklanmıştır.



Şekil 1.7.1. Data mirroring config ekranı

1.9. Sonuç

Bu projede, veritabanı yedekleme ve felaketten kurtarma süreçleri uygulamalı olarak gerçekleştirilmiştir. Farklı yedekleme türleri kullanılarak veri kaybı senaryoları test edilmiş ve çeşitli kurtarma yöntemleri başarıyla uygulanmıştır.

Elde edilen sonuçlar, doğru yapılandırılmış bir yedekleme stratejisinin veri güvenliği açısından kritik öneme sahip olduğunu göstermektedir. Ayrıca otomatik yedekleme, veri doğrulama ve ileri seviye kurtarma tekniklerinin sistem sürekliliği açısından önemli katkılar sağladığı görülmüştür.

2. VERİ TEMİZLEME VE ETL SÜREÇLERİ TASARIMI

2.1. Giriş

Bu çalışmada, büyük veri kümelerinin işlenmesi sürecinde kritik öneme sahip olan ETL (Extract, Transform, Load) süreçleri ele alınmıştır. Proje kapsamında, Microsoft SQL Server kullanılarak bir veri seti üzerinde veri temizleme, dönüştürme ve yükleme işlemleri gerçekleştirilmiştir.

Çalışmanın amacı; ham verilerde bulunan eksik, hatalı ve tutarsız değerlerin tespit edilmesi, uygun yöntemlerle düzeltilmesi ve analiz edilebilir hale getirilmesidir. Bu doğrultuda veri kalitesi artırılmış, dönüşüm işlemleri uygulanmış ve elde edilen sonuçlar çeşitli raporlar aracılığıyla değerlendirilmiştir.

Ayrıca süreç boyunca staging ve production katmanları kullanılarak profesyonel veri işleme mimarisi modellenmiştir.

2.2. Veri Seti ve Ortam Hazırlığı (Extract)

Çalışmada kullanılan veri seti, Kaggle platformundan temin edilmiştir. Veri seti, otel rezervasyonlarına ait bilgileri içermekte olup eksik ve hatalı veriler barındırmaktadır.

Proje kapsamında “HotelProject” adlı bir veritabanı oluşturulmuş ve veri seti bu veritabanına aktarılmıştır. Veri aktarımı sırasında karşılaşılan hatalar giderilmiş, veri tipleri uygun şekilde düzenlenmiş ve eksik verilerin kabul edilebilmesi için gerekli yapılandırmalar yapılmıştır.

booking_changes	deposit_type	agent	company	days_in_waiting_list	customer_type	adr	required_car_parking_spaces	total_of_special_requests	reservation_status	reservation_status_date
3	No Deposit	NULL	NULL	0	Transient	0	0	0	Check-Out	2015-07-01
4	No Deposit	NULL	NULL	0	Transient	0	0	0	Check-Out	2015-07-01
0	No Deposit	NULL	NULL	0	Transient	75	0	0	Check-Out	2015-07-02
0	No Deposit	304	NULL	0	Transient	75	0	0	Check-Out	2015-07-02
0	No Deposit	240	NULL	0	Transient	98	0	1	Check-Out	2015-07-03
0	No Deposit	240	NULL	0	Transient	98	0	1	Check-Out	2015-07-03
0	No Deposit	NULL	NULL	0	Transient	107	0	0	Check-Out	2015-07-03
0	No Deposit	303	NULL	0	Transient	103	0	1	Check-Out	2015-07-03
0	No Deposit	240	NULL	0	Transient	82	0	1	Canceled	2015-05-06
0	No Deposit	15	NULL	0	Transient	1055	0	0	Canceled	2015-04-22
0	No Deposit	240	NULL	0	Transient	123	0	0	Canceled	2015-06-23
0	No Deposit	240	NULL	0	Transient	145	0	0	Check-Out	2015-07-05
0	No Deposit	240	NULL	0	Transient	97	0	3	Check-Out	2015-07-05

Şekil 2.1.1. Düzenlenmemiş veri

2.3. Staging Katmanı Oluşturma

Veri işleme sürecinin sağlıklı bir şekilde yürütülebilmesi amacıyla staging (ara katman) tablosu oluşturulmuştur. Bu tablo, ham verilerin üzerinde değişiklik yapılmadan kopyalanarak işlenmesine olanak sağlamaktadır.

Bu kapsamda; veri setine uygun sütun yapısına sahip “Staging_HotelData” tablosu oluşturulmuştur, tüm sütunlar esnek veri işleme amacıyla NVARCHAR veri tipinde tanımlanmıştır, ana veri tablosundan staging tablosuna veri aktarımı gerçekleştirilmiştir. Ayrıca veri setinde birincil anahtar bulunmadığı için tabloya “RowID” adlı otomatik artan bir kimlik sütunu eklenmiştir.

```sql
CREATE TABLE Staging_HotelData (
    hotel NVARCHAR(MAX), is_canceled NVARCHAR(MAX), lead_time NVARCHAR(MAX),
    arrival_date_year NVARCHAR(MAX), arrival_date_month NVARCHAR(MAX),
    arrival_date_week_number NVARCHAR(MAX), arrival_date_day_of_month NVARCHAR(MAX),
    stays_in_weekend_nights NVARCHAR(MAX), stays_in_week_nights NVARCHAR(MAX),
    adults NVARCHAR(MAX), children NVARCHAR(MAX), babies NVARCHAR(MAX),
    meal NVARCHAR(MAX), country NVARCHAR(MAX), market_segment NVARCHAR(MAX),
    distribution_channel NVARCHAR(MAX), is_repeated_guest NVARCHAR(MAX),
    previous_cancellations NVARCHAR(MAX), previous_bookings_not_canceled NVARCHAR(MAX),
    reserved_room_type NVARCHAR(MAX), assigned_room_type NVARCHAR(MAX),
    booking_changes NVARCHAR(MAX), deposit_type NVARCHAR(MAX),
    agent NVARCHAR(MAX), company NVARCHAR(MAX), days_in_waiting_list NVARCHAR(MAX),
    customer_type NVARCHAR(MAX), adr NVARCHAR(MAX),
    required_car_parking_spaces NVARCHAR(MAX), total_of_special_requests NVARCHAR(MAX),
    reservation_status NVARCHAR(MAX), reservation_status_date NVARCHAR(MAX)
);
GO

INSERT INTO Staging_HotelData
SELECT * FROM [dbo].[hotel_bookings];

ALTER TABLE Staging_HotelData ADD RowID INT IDENTITY(1,1);
```

Şekil 2.2.1. Staging katmanı oluşturma

2.4. Veri Dönüştürme (Transform)

Veri dönüşüm işlemleri kapsamında staging tablosundaki veriler üzerinde çeşitli düzenlemeler yapılmıştır. Bu işlemler sonucunda “Production_HotelData” adlı yeni bir tablo oluşturulmuştur.

Gerçekleştirilen dönüşüm işlemleri aşağıdaki gibidir:

- Metin (string) formatındaki sayısal veriler uygun veri tiplerine dönüştürülmüştür
- Eksik (NULL) değerler uygun durumlarda varsayılan değerler (örneğin 0) ile değiştirilmiştir
- Ay isimleri standartlaştırılarak büyük harfe dönüştürülmüştür
- Tarih alanları uygun formatta birleştirilerek DATE tipine dönüştürülmüştür
- Günlük ücret (ADR) değeri ondalıklı (decimal) veri tipine çevrilmiştir
- Yetişkin, çocuk ve bebek sayıları toplanarak “total_guests” adlı yeni bir alan oluşturulmuştur

- “agent” sütunundaki hatalı ve eksik değerler temizlenerek “agent_id” olarak yeniden yapılandırılmıştır

Bu işlemler sonucunda veri seti analiz edilebilir, tutarlı ve standart bir yapıya dönüştürülmüştür.

```sql
DROP TABLE IF EXISTS Production_HotelData;
GO
SELECT
    RowID,hotel,
    CASE
        WHEN agent IS NULL OR agent = 'NULL' OR agent = '' THEN 0
        ELSE TRY_CAST(agent AS INT)
    END AS agent_id,

    CAST(adults AS INT) +
    COALESCE(TRY_CAST(children AS INT), 0) +
    CAST(babies AS INT) AS total_guests,

    TRY_CONVERT(DATE,
        arrival_date_year + '-' +
        arrival_date_month + '-' +
        arrival_date_day_of_month) AS arrival_date,

    CAST(is_canceled AS INT) AS is_canceled,
    COALESCE(TRY_CAST(lead_time AS INT), 0) AS lead_time,
    UPPER(arrival_date_month) AS arrival_month,
    TRY_CAST(adr AS DECIMAL(10,2)) AS daily_rate,

    arrival_date_year, arrival_date_day_of_month
INTO Production_HotelData
FROM Staging_HotelData;
```

Şekil 2.3.1. Dönüştürme işlemleri

2.5. Veri Yükleme (Load)

Dönüştürme işlemleri tamamlandıktan sonra temizlenen veriler “Production_HotelData” tablosuna aktarılmıştır. Bu yapı sayesinde; ham veri (source) korunmuştur, staging katmanında yapılan işlemler izole edilmiştir, temiz ve analiz edilebilir veri production katmanında saklanmıştır. Böylece ETL sürecinin “Load” aşaması başarıyla tamamlanmıştır.

```sql
INTO Production_HotelData
FROM Staging_HotelData;
```

Şekil 2.4.1. Veri yükleme

2.6. Veri Temizleme ve Kalite Analizi

Veri temizleme sürecinde, özellikle eksik ve hatalı veriler üzerinde yoğunlaşılmıştır.

Yapılan analizlerde özellikle “agent” sütununda yüksek miktarda eksik veri bulunduğu tespit edilmiştir. Bu eksik değerler uygun şekilde dönüştürülerek veri bütünlüğü sağlanmıştır. Veri kalitesini ölçmek amacıyla çeşitli SQL sorguları ile raporlar oluşturulmuştur. Bu raporlar sayesinde; eksik veri sayıları hesaplanmıştır, dönüşüm öncesi ve sonrası karşılaştırmalar yapılmıştır, veri temizleme işlemlerinin başarısı doğrulanmıştır. Örneğin, staging tablosunda bulunan NULL değerlerin production tablosunda tamamen ortadan kaldırıldığı gözlemlenmiştir.

```sql
DROP TABLE IF EXISTS Production_HotelData;
SELECT
    RowID,
    hotel,
    CASE
        WHEN agent IS NULL OR agent = 'NULL' THEN 0
        ELSE TRY_CAST(agent AS INT)
    END AS agent_id
INTO Production_HotelData
FROM Staging_HotelData;
```

Şekil 2.5.1. Eksik verinin temizlenmesi

```sql
SELECT COUNT(*) AS Null_Agent_Count_After
FROM Production_HotelData
WHERE agent_id IS NULL;
```

91 % No issues found

Results Messages

	Null_Agent_Count_After
1	0

Şekil 2.5.2. Agent sütununun eksik veri sayısı için veri raporu

2.7. Karşılaştırmalı Raporlama

Veri temizleme sürecinin etkisini göstermek amacıyla staging ve production tabloları arasında karşılaştırmalar yapılmıştır. Bu kapsamda; ham verilerdeki eksik değerler ile temizlenmiş verilerdeki karşılıkları karşılaştırılmıştır, NULL değerlerin uygun şekilde dönüştürüldüğü doğrulanmıştır, örnek kayıtlar üzerinden dönüşüm işlemleri gözlemlenmiştir. Ayrıca, join işlemleri kullanılarak aynı verinin ham ve temizlenmiş halleri yan yana gösterilmiş ve dönüşümün doğruluğu analiz edilmiştir.

```sql
SELECT TOP 10
    S.RowID,
    S.agent AS Raw_Value,
    P.agent_id AS Cleaned_Value
FROM Staging_HotelData S
JOIN Production_HotelData P ON S.RowID = P.RowID
WHERE S.agent IS NULL OR S.agent = 'NULL';
```

91 % No issues found

Results | Messages

	RowID	Raw_Value	Cleaned_Value
1	146	NULL	0
2	1	NULL	0
3	314	NULL	0
4	5004	NULL	0
5	75	NULL	0
6	5398	NULL	0
7	1008	NULL	0
8	317	NULL	0
9	6006	NULL	0
10	751	NULL	0

Şekil 2.6.1. Farklı tablolar için veri raporu karşılaştırması

2.8. Sonuç

Bu projede, ETL süreçleri kullanılarak bir veri seti üzerinde kapsamlı veri temizleme ve dönüştürme işlemleri gerçekleştirilmiştir.

Ham veriler staging katmanına alınmış, burada çeşitli dönüşümler uygulanmış ve temizlenen veriler production katmanına aktarılmıştır. Süreç boyunca oluşturulan veri kalitesi raporları sayesinde yapılan işlemler doğrulanmış ve veri bütünlüğü sağlanmıştır.

Elde edilen sonuçlar, ETL süreçlerinin veri analizi ve karar destek sistemleri için kritik öneme sahip olduğunu göstermektedir. Özellikle veri temizleme ve standartlaştırma adımlarının, güvenilir ve doğru analizler yapılabilmesi açısından vazgeçilmez olduğu görülmüştür.